

Asynchronous Managed Pointer for Heterogeneous and Distributed Computing

Revision History

P0567r0 -> P0567r1:

- A new section has been added to describe the requirements of the paper.
- Corrections and updates have been made to the example code.
- The synopsis has been extended to cover the requirements for the **executor** and **future** types.
- Restriction on having only one device within a system capable of performing synchronisation operations has been removed.
- Implicit synchronisation removed as future work for now as it requires static reflection.
- The `managed_ptr` now interfaces with an **executor** rather than an **execution context**.
- The `managed_ptr` now has an **executor** associated with it which can be retrieved with the member function `get_local_execution_context()`.
- The `managed_ptr` synchronization member function `get()` and the **executor** class synchronization member functions `get()`, `then_get()`, `put()` and `then_put()` have been removed, leaving only the global synchronization functions `get()`, `then_get()`, `put()` and `then_put()`.
- The `managed_ptr` no longer has a size associated with it, instead it represents a pointer to a single value in the same way a `std::shared_ptr` does.
- The `managed_ptr` member alias `future_type` has been removed and the global synchronization functions `get()`, `then_get()`, `put()` and `then_put()` now return `executor_future_t`.
- The `managed_ptr` constructors have been altered so there is now only two constructors, one that initialises with an allocator, that is defaulted to the allocator associated with the associated **executor** and one that takes a raw pointer.
- The `managed_ptr` pointer and dereference operators have been specialised for const instantiations.
- The `managed_ptr` now has the member function `is_allocated()` to query if the `managed_ptr` has been allocated on a particular execution context.
- The compile-time traits `can_put_v`, `can_get_v`, `can_sync_v`, `can_allocate_v` and `can_deallocate_v` have been added for querying an executor's capabilities with regard to data movement.
- The `managed_ptr` member function `is_accessible` now takes an **executor** parameter with a default argument allowing you to query if the `managed_ptr` instance is accessible in another execution context.
- The global synchronization functions `get()` and `then_get()` no longer take an **executor** parameter, as the associated **executor** can be inferred from the `managed_ptr` parameters.
- The global synchronization operations `sync` and `then_sync` which take an **execution channel** have been added.
- Add explicit allocation and deallocation via global allocation functions `allocate`, `then_allocate`, `deallocate` and `then_deallocate`.

Feedback from P0567r0, 2017-01-30

Much valuable feedback was given since the last revision of this paper, this section will identify the issues that were raised and points to their respective solutions.

Read-only pointers

It was suggested that the `managed_ptr` should be capable of providing read-only access, where write access is restricted, this can be very useful as it allows an implementation to make use of read-only partitions of memory which are often more efficient to access. This is now supported in this paper and is described in section 3.3.

Allocation of managed memory

There were many questions regarding how the how and when memory is allocated for a `managed_ptr` instance and what happens on an allocation failure. Section 3.3.4 has been added which describes these behaviours.

Optimising synchronization between execution contexts

It was suggested that it be possible for an implementation to optimise synchronisation of a `managed_ptr` between specific executors rather than having to synchronise via a host executor. A solution that was being discussed but didn't make it into the first revision of the paper was the concept of channels. This is now supported in this paper and is described in section 3.5.

Discuss memory access in terms of side effects

It was suggested that this proposal should describe memory access in terms of visible sided effects rather than caching. This has been introduced as part of the requirement for supporting different memory architectures and the paper has been updated to reflect this.

Host based and remote execution contexts

It was asked what would happen if `malloc()` were to be called on a GPGPU. The answer to this is that dynamic allocation cannot be supported for the general case as many GPUs do not support this due to hardware limitations. There are some GPUs which do support dynamic allocation such as devices supporting CUDA 2.x and above, however, this is not the common case. This question of *execution context* capabilities is described in section 3.3.5.

Supporting systems with unified address spaces

It was asked if the `managed_ptr` could allow concurrent access to memory from different execution contexts, this can be very important for systems which have regions of memory shared across discrete devices. This was discussed however did not make it into the first draft of the paper as a solution had not yet been reached. The current proposed solution is described in section 4.1 as future work.

1. Introduction

1.1. Summary

This paper proposes an extension to the unified interface for execution proposal [1] to facilitate the management and synchronisation of a memory allocation across multiple memory region(s). This addition is in the form of the class template `managed_ptr`; similar to the `std::shared_ptr` but with the addition that it can share its memory allocation across the memory region of it's host executor where it is constructed and the memory region(s) of one or more other executors.

The aim of this paper is to begin an exploratory work into designing a unified interface for data movement. There are many different architectures and data flow models to consider when designing such an interface so it is expected that this paper will serve only as a starting point.

1.2. Motivation

In non-heterogeneous or non-distributed systems, there is typically a single device: a CPU with a single memory region or an area of addressable memory. In contrast, heterogeneous and distributed systems have multiple devices with shared or discrete memory regions.

There are several key points to consider when addressing heterogeneous and distributed systems:

- Each device within a heterogeneous or distributed system can potentially require code compiled for a different ISA.
- Each device within a heterogeneous or distributed system can have shared or discrete regions of memory.
- Each heterogeneous or distributed systems can have different forms of communications between the devices; including shared memory, bus or network.
- Each device within a heterogeneous or distributed system can have varying levels of functionality; not all devices can function as

fully featured C++ threads.

As the act of dispatching work to a remote device to be executed was once only a problem for third party APIs and so too was the act of moving data to those devices for computations to be performed on. However, now that C++ is progressing towards a unified interface for execution [1] the act of moving data is now a problem for C++ to solve as well. The act of moving data to a remote device is very tightly coupled with the work being performed on said remote device. This means that this unified interface for data movement must also be tightly coupled with the unified interface for execution.

For a more in-depth analysis of the requirements for supporting heterogeneous systems in C++ see P0687r0: Data Movement in C++ [13].

The managed memory class will represent a memory allocation that may be available in one of many memory regions within a system throughout a program's execution. For the purposes of this paper, we will refer to such a memory allocation as a managed memory allocation as it describes a memory allocation that is accessible consistently across multiple memory regions. With this requirement comes the possibility that a memory region on a given device may not have the most recently modified copy of a managed memory allocation, therefore requiring synchronisation to move the data.

1.3. Influence

This proposal is influenced by many heterogeneous and distributed programming models; including SYCL [2], HPX [3], Kokkos [4], Raja [5], HAM-Offload [9] and MYO [11].

This approach is also heavily influenced by the proposal for a unified interface for execution as the interface proposed in this paper interacts directly with this.

2. Requirements / Design Goals

When attempting to standardise a unified interface for data movement there are many different aspects which need to be considered. These come from both the various architectures that this interface aims to abstract and the various data flow models which this interface aims to provide.

1. Must support different memory architectures
2. Must support different levels of synchronisation
3. Must support different data flow models
4. Must integrate seamlessly with executors
5. Must be interoperable between executor implementations
6. Must allow optimisation of data movement between devices
7. Must abstract device capabilities

This paper in it's current state aims to satisfy [1], [3], [4], [5], [6], [7], while [2] is still being developed.

2.1. Must support different memory architectures

In non-heterogeneous or non-distributed systems, there is typically a single device: a host CPU with a single memory region or an area of addressable memory. In contrast, heterogeneous and distributed systems have multiple devices (including the host CPU) often with their own discrete memory regions. A device in this respect can be any architecture that exists within a system that is C++ programmable; this can include CPUs, GPGPUs, APUs, FPGAs, DSPs, NUMA nodes, I/O devices and other forms of accelerators.

These memory regions can be categorised into two groups.

- The first group represents all the memory regions of systems with a single unified address space, that is a single area of addressable memory that is accessible across multiple devices. This can optionally be supported in hardware by devices having the same physical memory or in software by having a cache coherency layer accessing a shared virtual address space.
- The second group represents all the memory regions of systems with multiple address spaces, therefore having separate discrete areas of addressable memory, often in different physical memory.

A unified interface needs to be capable of supporting both of these kinds of memory architectures in such a way that any requirements for cache coherency is abstracted by the interface and associated coherency model.

2.2. Must support different levels of synchronisation

Different memory architectures, both unified and discrete; may support granularity at different levels. This can either be at the coarse-grained level; where synchronization between memory regions is performed at the execution function level, or this can be at the fine-grained level; where synchronization between memory regions is performed at the instruction level, allowing access to memory from different devices concurrently, using atomics to specify read and write order.

A unified interface needs to be capable of supporting both these levels of synchronisation.

2.3. Must support different data flow models

There are many different ways for a programming model to represent the data required for a computation and how that data is made available on a particular device. This paper will use the term **data-flow model** to refer to a model which describes what a computation's data dependencies are, where they are required to be accessed and when they are to be made available.

This paper identifies four kinds of data-flow models which this proposal must be capable of supporting. These are referred to as **explicit data-flow models**, **chained data-flow models**, **implicit data-flow models** and **graph-based data-flow models**.

An **explicit data-flow model** is one which allows users to manually specify the point at which a data dependency is made available to a computation via an API. These APIs can generally be synchronous or asynchronous, providing some form of synchronisation primitive which can be used to signal when the data dependency is available on the target device. An example of this could be an explicit copy operation:

```
copy(ptr, contextA, contextB);
execute(contextB, func(ptr));
```

A **chained data-flow model** is an extension of an explicit data-flow mode, where the synchronisation primitives also provide the capability of specifying further data dependency requirements or launching a computation when the data dependency is available on the target device. This kind of data-flow models are often coupled very tightly with the computation. An example of this could be future based continuations:

```
copy(ptr, contextA, contextB)
  then_execute(contextB, func(ptr));
```

An **implicit data-flow model** is one which allows data dependencies to be described by simply passing those data dependencies to a computation and having them implicitly made available on the target device. This kind of data-flow models can feel very natural for users, however, they can involve some with limitations to some data-flow optimisation such as prefetching or double buffering. An example of this could be a pointer that can be passed directly to a computation:

```
execute(contextB, func(ptr));
```

A **graph-based data-flow** is one which allows data dependencies to be described by a high-level graph; often in some form of DSL or other graph format. This kind of data-flow models, as they have access to a greater amount of information can often perform optimisations to the data dependency representation. As with the chained data-flow model the graph-based data-flow models are also coupled tightly with computation. An example of this could be a compile-time DSL that evaluates the parameters of an expression:

```
eval(contextB, y = a * x + y);
```

The proposal must provide an interface which can provide a foundation for all four of these data-flow models.

2.4. Must integrate seamlessly with executors

As data movement is tightly coupled with execution, it is necessary for a unified interface for data movement to be integrated with the unified interface for execution. This means such a unified interface must integrate seamlessly with the current design philosophy of executors. It must be compatible with the range of control structures which the executor abstraction supports and must support the range of platforms which executors can target.

At the same time, a unified interface for data movement must exist at a high level of abstraction and must avoid introducing too many requirements on executor implementations, in order to avoid introducing complex requirements for heterogeneous or distributed memory architectures into C++.

2.5. Must be interoperable between executor implementations

The unified interface for execution will have many different implementations which target different platforms and therefore a unified interface for data movement would too require this range of implementations.

It's important therefore that a unified interface for data movement be interoperable between different executor implementations, it must be possible to synchronise data between the memory region of one executor implementation and another.

2.6. Must allow optimisation of data movement between devices

A unified interface for data movement must provide interoperability between implementations, however, this will likely be inefficient as it will likely require a middle man stage that facilitates the required operations of each implementation.

Therefore it's important that a unified interface for data movement provide a facility for customising data movement between particular memory regions in order to optimise those operations rather than relying on a generic middle man.

2.7. Must abstract device capabilities

Many heterogeneous architectures such as GPGPUs, DSPs or FPGAs have considerable restrictions on the code that is executed on the compared to that of a CPU, such as no capability to dynamically allocate memory or signal directly with other devices during execution and the level of forward progress guarantees the device can provide. This means that a unified interface for data movement needs to take into consideration these restrictions in the design of how memory on devices is allocated and synchronised.

A typical example of this is that a CPU cluster would have no problem allocating a `managed_ptr` on any one of its nodes and performing synchronisation operations on any node, whereas with a typical GPU all allocation and synchronisation must be done from a host CPU device prior to execution.

3. Proposed Additions

3.1. Header `<execution>` synopsis

```
namespace std {
namespace experimental {
inline namespace concurrency_v2 {
namespace execution {

/* extensions to executor classes */
class executor {
    ...

    /* aliases */
    using executor_memory_allocator_type = __executor_memory_allocator_type__;

    /* member functions */
    executor_memory_allocator_type executor_memory_allocator() const;

    ...
};

/* managed_ptr class template */
template <typename T>
class managed_ptr {
public:

    /* aliases */
    using value_type          = __undefined_attributes__ T;
    using pointer              = value_type *;
```

```

using const_pointer      = const value_type *;
using reference          = value_type &;
using const_reference    = const value_type &;
using host_executor_type = __host_executor_type__;
using executor_allocator_type = host_executor_type::executor_memory_allocator_type;

/* constructors */
managed_ptr(pointer); // (1)
template <typename Allocator>
managed_ptr(Allocator allocator = executor_allocator_type{}); // (2)

/* special member functions */
managed_ptr(const managed_ptr &);
managed_ptr(managed_ptr &&);
managed_ptr &operator=(const managed_ptr &);
managed_ptr &operator=(managed_ptr &&);
~managed_ptr();

/* member functions */
host_executor_type &executor() const;
template <typename Executor>
bool is_allocated(Executor = host_executor_type{}) const;
template <typename Executor>
bool is_accessible(Executor = host_executor_type{}) const;

/* operators of const T instantiation */
const_reference operator*() const;
const_pointer operator->() const;

/* operators of non-const T instantiation */
reference operator*() const;
pointer operator->() const;
};

/* extended requirements for executor_future_t */
template <typename T>
class executor_future_t {
    ...

    /* synchronization member functions */
    template <typename Executor, typename... ManagedPtrN>
    future_type_t<void> put(Executor, ManagedPtrN...);
    template <typename... ManagedPtrN>
    future_type_t<void> get(ManagedPtrN...);
    template <typename SynchronizationChannel, typename SrcExecutor,
              typename DestExecutor, typename... ManagedPtrN>
    future_type_t<void>
    sync(SynchronizationChannel = __default_synchronization_channel__{},
        SrcExecutor, DestExecutor, ManagedPtrN...);

    /* allocation member functions */
    template <typename Executor, typename... ManagedPtrN>
    future_type_t<void> allocate(Executor, ManagedPtrN...);
    template <typename Executor, typename... ManagedPtrN>
    future_type_t<void> deallocate(Executor, ManagedPtrN...);

    ...
};

```

```

/* synchronization functions */
template <typename Executor, typename... ManagedPtrN>
future_type_t<void> put(Executor, ManagedPtrN...);
template <typename Executor, typename Predicate, typename... ManagedPtrN>
future_type_t<void> then_put(Executor, Predicate, ManagedPtrN...);
template <typename... ManagedPtrN>
future_type_t<void> get(ManagedPtrN...);
template <typename Predicate, typename... ManagedPtrN>
future_type_t<void> then_get(Predicate, ManagedPtrN...);
template <typename SynchronizationChannel, typename SrcExecutor,
          typename DestExecutor, typename... ManagedPtrN>
future_type_t<void>
sync(SynchronizationChannel = __default_synchronization_channel_{},
     SrcExecutor, DestExecutor, ManagedPtrN...);
template <typename SynchronizationChannel, typename SrcExecutor,
          typename DestExecutor, typename Predicate, typename... ManagedPtrN>
future_type_t<void>
then_sync(SynchronizationChannel = __default_synchronization_channel_{},
          SrcExecutor, DestExecutor, Predicate, ManagedPtrN...);

/* allocation functions */
template <typename Executor, typename... ManagedPtrN>
future_type_t<void> allocate(Executor, ManagedPtrN...);
template <typename Executor, typename Predicate, typename... ManagedPtrN>
future_type_t<void> then_allocate(Executor, Predicate, ManagedPtrN...);
template <typename Executor, typename... ManagedPtrN>
future_type_t<void> deallocate(Executor, ManagedPtrN...);
template <typename Executor, typename Predicate, typename... ManagedPtrN>
future_type_t<void> then_deallocate(Executor, Predicate,
                                   ManagedPtrN...);

/* executor traits */
template<class T> struct can_put;
template<class T> struct can_get;
template<class T> struct can_sync;
template<class T> struct can_allocate;
template<class T> struct can_deallocate;
template<class T> constexpr bool can_put_v = can_put<T>::value;
template<class T> constexpr bool can_get_v = can_get<T>::value;
template<class T> constexpr bool can_sync_v = can_sync<T>::value;
template<class T> constexpr bool can_allocate_v = can_allocate<T>::value;
template<class T> constexpr bool can_deallocate_v = can_deallocate<T>::value;

} // namespace execution
} // namespace concurrency_v2
} // namespace experimental
} // namespace std

```

Figure 1: Execution synopsis

3.2. Requirements on Executor and Execution Context

In order to facilitate the `managed_ptr`, it is necessary to introduce requirements on the *executor* and the *execution context*; the class associated with an *executor* which encapsulates the underlying execution resource on which *execution agents* are executed.

3.2.1. Execution Context Memory Region

In addition to a number of *execution agents*, an *execution context* must also encapsulate at least one memory region that is concurrently accessible by all *execution agents* of any single invocation of an execution function. It is not required for the memory region to be

concurrently accessible to *execution agents* of another invocation of an execution function or *execution agents* executing on a different *execution context*.

This requirement currently satisfies the coarse-grained level of synchronisation described in requirement 2. These requirements will be loosened in a later iteration of this paper when a suitable interface for fine-grained synchronisation is included. The current proposal for this is described in section 4.1 as future work.

3.2.2. Executor Memory Allocator

The *execution context* must also provide an alias to an allocator type called `executor_memory_allocator_type` that is capable of allocating memory in it's memory region, and a member function for retrieving an instance of this type called

```
executor_memory_allocator()
```

3.3. Managed Pointer Class Template

The proposed addition to the standard library is the `managed_ptr` class template. The `managed_ptr` is a smart pointer which has ownership of a virtual allocation of contiguous memory that can be shared between any number of devices via a set of synchronisation operations.

The `managed_ptr` class template has a single template parameter; `T` specifying the type of the pointer.

The type parameter `T` specifies the pointer type, any `T` satisfies the `managed_ptr` pointer type requirements if:

Another solution for read-only access was considered where access to a `managed_ptr` instance could be dynamically specified via properties using the proposed `array_ref` interface [10]. However, it was felt that this approach would be too verbose and would introduce unnecessary overhead for cases which did not require the properties to be specified. Additionally, as many implementations would wish to make use of read-only partitions of memory this could require an implementation to copy in an out of this memory partition dynamically which could introduce opaque latency in synchronisation.

- `T` is a standard layout type.
- `T` is trivially copy-constructible.

If `T` is a const type then the `managed_ptr` is considered to be read-only. This allows implementations to optimise the allocation of memory in read-only segments of the memory region. Additionally, the de-reference and pointer operators return a const pointer and reference respectively preventing writing to the memory.

3.3.1. Construction and destruction

A `managed_ptr` can be constructed in two ways:

- for constructor (1) the `managed_ptr` takes an existing pointer and does not allocate memory.
- for constructor (2) the `managed_ptr` allocates using the allocator specified by the `allocatorT` parameter, which defaults to the allocator of the associated *executor* if one is not provided.

The default constructor for `T` must not be called by the `managed_ptr` during construction of the `managed_ptr`.

The destructor must not perform any synchronisation operations.

3.3.2. Execution context capabilities

In order to support a wide range of systems, both heterogeneous and distributed, a high-level abstraction for the `managed_ptr` is necessary, however not all devices will be able to support all of a `managed_ptr`'s capabilities.

As the `managed_ptr` can be used on different devices it can have different capabilities depending on where it is currently being used. The `managed_ptr` will likely be compiled separately or at least have different restrictions imposed on the code that is executed depending on where it is used. It is important to be able to determine what capabilities the `managed_ptr` supports where it is being used.

These capabilities can be queried via a series of compile-time traits. These are `can_put_v`, `can_get_v`, `can_sync_v`, `can_allocate_v` and `can_deallocate_v`. Each of these compile time traits can be used to query whether a particular executor class supports the synchronisation or allocation function of the same name.

These traits are currently limited to the synchronisation and allocation functions however this can be extended to include queries for other capabilities.

3.3.3. Memory accessibiity

During the lifetime of a `managed_ptr` it's managed memory allocation can exist in the memory region(s) associated with any number of *execution contexts*, but may only be accessible in one of these memory regions at any given time. The *execution context* who's memory region is currently accessible is said to be the accessible *execution context*.

This limited may be relaxed in a future version of the paper when a `managed_ptr` can be concurrently accessed in more than one memory region. In this case, there could be more than one accessible memory region. This is further described in section 4.1.

The `managed_ptr` must maintain a reference to the current accessible *execution context*.

The associated *execution context* and *executor* of a `managed_ptr` can also be accessible, the `managed_ptr` has a member function for querying wether the associated *execution context* and *executor* is accessible; `is_accessible()` .

The data at the memory region of the `managed_ptr` s associated *execution context* and *executor* can be read and modified (if the type `T` is a non-const type) via the de-reference and pointer operators.

3.3.4. Resource aquisition

The `managed_ptr` must allocate memory within the memory region of any execution context which that `managed_ptr` is accessible on via that execution context's allocator. This does not have to be in tandem or on construction of the `managed_ptr` instance, the point at which allocation happens for each execution context is implementation defined, the only requirement is that the memory must be allocated (if not already so) prior to the `managed_ptr` being accessed on said execution context. Additionally, the `managed_ptr` must deallocate in all memory regions that the `managed_ptr` instance has been allocated on, during its destructor.

If a user wishes to explicitly allocate or deallocate memory for a `managed_ptr` instance on a particular execution context, this can be done by calling the allocation operations `allocate` , `then_allocate` , `deallocate` and `then_deallocate` . Each of which is an asynchronous operation which performs an allocation or allocation in a particular execution context and returns a future to the completion of the operation.

If any memory allocation or deallocation operations fails, then an exception is thrown and stored in the future object that is returned by the operations which triggered the allocation operation.

3.3.5. Synchronization

For an **execution context** to be the accessible **execution context**, if it is not already, a synchronisation operation is required. A synchronisation operation is an implementation defined asynchronous set of commands which moves the data from the currently accessible memory region to another memory region which returns a future object that can be used to wait for the operation to complete. From the point at which a synchronisation operation is triggered the currently accessible memory region is no longer accessible. Once the synchronisation point is complete (i.e. the future returned from the synchronisation operation is complete) the memory region the data is being synchronised to is now the accessible memory region.

Synchronisation operations are coarse-grained synchronisation in that they synchronise the entire managed memory allocation of a `managed_ptr` .

This limitation may be relaxed in a future version of the paper when a `managed_ptr` can be concurrently accessed in more than one memory region. In this case, there synchronisation could be performed at smaller granularities using mapping operations or atomic operations. This is further described in section 4.1.

There are three synchronisation operations; `get` , `put` and `sync` , each taking one or more `managed_ptr` parameters. These synchronisation operations perform the following synchronisation for each `managed_ptr` parameter:

- The `get` operation synchronises the data at the memory region at a particular *executor* to the memory region at the *executor* associated with the `managed_ptr` .
- The `put` operation synchronises the data at the memory region at the *executor* associated with the `managed_ptr` to the memory region at a particular *executor*.
- The `sync` operation synchronises the data at the memory region at one particular *executor* to the memory region at another particular *executor*.

In the previous revision of this paper R0, it was also possible to implicitly trigger synchronisation operations by simply passing a `managed_ptr` to a control structure such as `async`. However, this has been removed due to a limitation that may affect implementations. This is described in section 4.2.

There are a further three synchronisation operations; `then_get`, `then_put` and `then_sync`, which are equivalent to `get`, `put` and `sync` but take a predicate future parameter that the operation must wait on before executing. These are used to provide a continuations interface for synchronisation operations. Equivalent operations to these are also required as member functions on the future type. This is described further in section 3.4.

If any synchronisation operations fail, then an exception is thrown and stored in the future object that is returned by the operations which triggered the synchronisation operation.

Below is an example of using the `put` and `get` synchronisation operations to synchronise data to and from a GPU:

```
using std::experimental;

/* Define the data structure to share across devices. */
struct my_data { /* ... */ };

/* Construct a context and executor for executing work on the GPU */
gpu_execution_context gpuContext;
auto gpuExecutor = gpuContext.executor();

/* Retrieve gpu allocator */
auto gpuAllocator = gpuContext.allocator();

/* Construct a managed_ptr ptrA allocated on the memory region of the host executor */
execution::managed_ptr<my_data> ptrA;

/* Construct a managed_ptr ptrB allocated on the memory region of the GPU executor */
execution::managed_ptr<my_data> ptrB(gpuAllocator);

/* Populate ptrA */
populate(ptrA);

/* Construct a series of compute and data operations */
auto fut =
    execution::put(gpuExecutor, ptrA)
        .then_async_execute(gpuExecutor, [=]() { /* ... */ })
        .then_get(ptrB);

/* Wait on the operations to execute */
fut.wait();

/* Print the result */
print(ptrB);
```

Figure 2: Example of synchronization operations

3.4. Requirements on the Future Type

The executor future type `executor_future_t` that is either the result of or calls a synchronisation continuation must provide the member functions `put`, `get`, `sync`, `allocate` and `deallocate` as described in the execution synopsis (Figure 1).

3.5. Optimising Synchronisation Operations using Synchronisation Channels

Synchronisation channels are objects which represent a connection between two execution contexts, for example between two remote devices or between an I/O device and a remote device. The default synchronisation channel which supports synchronisation between all execution contexts is that which synchronises via the host executor. However, implementers can provide their own synchronisation

channels which can be used by the user in synchronisation operations to optimise them.

For example:

```
using namespace std::experimental::concurrency_v2::execution;

struct my_data { /* ... */ };

vendor_x::input_stream_execution_context inputStreamContext;
vendor_x::gpu_execution_context gpuContext;
vendor_x::input_stream_to_gpu_channel inputStreamToGPUChannel;

auto inputStreamExec = inputStreamContext.executor();

auto gpuExec = gpuContext.executor();

managed_ptr<my_data> ptr(inputStreamContext.executor_memory_allocator());

for (;;) {
    auto fut =
        execute(inputStreamExec, [=]() { read_input(ptr); })
        .then_sync(inputStreamToGPUChannel, inputStreamExec, gpuExec)
        .then_execute(gpuExec, [=]() { process(ptr); })
        .post_process(fut.get());
}
```

Figure 3: Synchronization Channels Example

This is not the most efficient example, it could be improved by double buffering the synchronisation and computation, though it demonstrates how channels can be used to avoid synchronising via a host executor.

4. Future Work

There are many other considerations to make when looking at a model for data movement for heterogeneous and distributed systems, however, this paper aims to establish a foundation which can be extended to include other paradigms in the future.

4.1. Shared Accessibility

Systems which support a unified address space (i.e. sharing a single memory region across different *execution contexts*) could allow for a `managed_ptr` instance to share accessibility between multiple *execution contexts*. This would allow a `managed_ptr` to be concurrently accessible by multiple *execution contexts* via the use of atomics providing those *execution contexts* share a memory region.

The current proposed solution is that the `managed_ptr` have an additional template parameter specifying the `memory_mode`, allowing it to be specialised for supporting different memory architectures. The `memory_mode` of a `managed_ptr` can be either `memory_mode::discrete`, `memory_mode::unified` or `memory_mode::shared`:

- The memory mode `memory_mode::discrete` can only have a single accessible execution context so cannot be accessed concurrently and has discrete memory regions so cannot support passing pointer-based structures.
- The memory mode `memory_mode::unified` can only have a single accessible execution context so cannot be accessed concurrently but has a unified memory region so can support passing pointer-based structures.
- The memory mode `memory_mode::shared` can have multiple accessible execution contexts so can be accessed concurrently via the use of atomics and has a unified memory region so can support passing pointer-based structures.

This idea has not yet been integrated into the rest of the proposal and is considered future work as the design has not yet been fully fleshed out. There are other factors that have to be considered such as the execution context - memory region topology and atomic operations.

4.2. Implicit Synchronization

In a previous revision of this paper R0, it was possible to trigger implicit synchronisation operations by simply passing a `managed_ptr` to a control structure such as `async`. However, this has been removed due to limitations that may affect an implementation. As the current executor interface does not support variadic execution function parameters, an implementation would be required to trigger synchronisation operations on all control structures for `managed_ptr` captures. Firstly this may incur unwanted overhead and secondly, this could potentially require static reflection for an optimal implementation. For these reasons this feature has been removed and instead kept as a potential future work that could be incorporated if proven to be implementable without creating overhead.

4.3. Additional Containers

We may wish to extend this principle of a managed pointer to other containers that would be useful to share across heterogeneous and distributed systems such as vectors or arrays. This could be done by having containers such as `managed_vector` or `managed_array` that would have similar requirements to the standard containers of the same names in terms of storage and access yet would be extended to support access from remote devices as with the `managed_ptr`.

4.4. Hierarchical Memory Architectures

While CPUs have a single flat memory region with a single address space, most heterogeneous devices have a more complex hierarchy of memory regions each with their own distinct address spaces. Each of these memory regions has a unique access scope, semantics and latency. Some heterogeneous programming models provide a unified or shared memory address space to allow more generic programming such as OpenCL 2.x [6], HSA [7] and CUDA [8], however, this will not always result in the most efficient memory access. This can be supported either in hardware where the host CPU and remote devices share the same physical memory or software where a cross-device cache coherency system is in place, and there are various different levels at which this feature can be supported. In general, this means that pointers that are allocated in the host CPU memory region can be used directly in the memory regions of remote devices, though this sometimes requires mapping operations to be performed.

We may wish to investigate this feature further to incorporate support for this kind of systems, ensuring that the `managed_ptr` can fully utilise the memory regions on these systems.

Appendix A: Naming considerations

During the development of this paper, many names were considered both for the `managed_ptr` itself and for its interface.

Alternative names that were considered for `managed_ptr` were `temporal` as it described a container which gave temporal access to a managed memory allocation, `managed_container` as the original design was based on the `std::vector` container and `managed_array` as the managed memory allocation is statically sized.

Alternative names for the `put()` and `get()` interface were `acquire()` and `release()` as you were effectively acquiring and releasing the managed memory allocation and `send()` and `receive()` as you are effectively sending and receiving back the managed memory allocation.

References

[1] P0443R1 A Unified Executors Proposal for C++:
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0443r1.html>

[2] SYCL 1.2 Specification:
<https://www.khronos.org/registry/sycl/specs/sycl-1.2.pdf>

[3] STELLAR-GROUP HPX Project:
<https://github.com/STELLAR-GROUP/hpx>

[4] Kokkos Project:
<https://github.com/kokkos>

[5] Raja Project:
<http://software.llnl.gov/RAJA/>

[6] OpenCL 2.2 Specification

<https://www.khronos.org/registry/OpenCL/specs/opencvl-2.2.pdf>

[7] HSA Specification

<http://www.hsafoundation.com/standards/>

[8] CUDA Unified Memory

<https://devblogs.nvidia.com/parallelforall/unified-memory-in-cuda-6/>

[9] HAM-Offload Programming Model for Distributed Systems

<https://github.com/noma/ham>

[10] Polymorphic Multidimensional Array Reference

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0009r3.html>

[11] MYO Programming Model (Mine Yours Ours)

<http://pleiades.ucsc.edu/doc/intel/mic/myo/tutorials/README.txt>

[12] Programming for Multicore and Many-core Products including Intel® Xeon® processors and Intel® Xeon Phi™ X100 Product Family coprocessors

<https://software.intel.com/en-us/articles/programming-for-multicore-and-many-core-products>

[13] P0687r0 Data Movement in C++

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/P0687r0.html>